

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

”Сибирский государственный университет телекоммуникаций и
информатики”

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

Рабочий дневник по дисциплине
Моделирование мобильных систем

Студент:
Группа ИА-331

И.А. Иванов

Преподаватель:
Р.В. Ахпашев

Новосибирск 2026 г.

СОДЕРЖАНИЕ

1	ПРАКТИКА 1. ЗНАКОВОЕ КОДИРОВАНИЕ	4
1.1	Задание.....	4
1.2	Теоретическое описание	4
1.3	Реализация.....	4
1.4	Результат.....	5
1.5	Вывод	6
2	ПРАКТИКА 2. ПОМЕХОУСТОЙЧИВОЕ КОДИРОВАНИЕ	7
2.1	Задание.....	7
2.2	Теоретическое описание	7
2.3	Реализация.....	7
2.4	Результат.....	9
2.5	Вывод	9
3	ПРАКТИКА 3. ПЕРЕМЕЖЕНИЕ	10
3.1	Задание.....	10
3.2	Теоретическое описание	10
3.3	Реализация.....	10
3.4	Результат.....	11
3.5	Вывод	11
4	ПРАКТИКА 4. QPSK МОДУЛЯЦИЯ	12
4.1	Задание.....	12
4.2	Теоретическое описание	12
4.3	Реализация.....	12
4.4	Результат.....	13
4.5	Вывод	13
5	ПРАКТИКА 5. OFDM МОДУЛЯЦИЯ	14
5.1	Задание.....	14
5.2	Теоретическое описание	14
5.3	Реализация.....	14
5.4	Результат.....	16

5.5	Вывод	16
6	ПРАКТИКА 6. МОДЕЛЬ КАНАЛА ПЕРЕДАЧИ.....	17
6.1	Задание.....	17
6.2	Теоретическое описание	17
6.3	Реализация.....	17
6.4	Результат	18
6.5	Вывод	18
7	ПРАКТИКА 7. ЭКВАЛАЙЗИРОВАНИЕ И OFDM ДЕМОДУЛЯЦИЯ	19
7.1	Задание.....	19
7.2	Теоретическое описание	19
7.3	Реализация.....	19
7.4	Результат	20
7.5	Вывод	20
8	ПРАКТИКА 8. РАСЧЕТ BER И ПОСТРОЕНИЕ ГРАФИКОВ	21
8.1	Задание.....	21
8.2	Теоретическое описание	21
8.3	Реализация.....	21
8.3.1	Расчет BER	21
8.3.2	Построение графиков	21
8.4	Результат	23
8.5	Вывод	23

ПРАКТИКА 1. ЗНАКОВОЕ КОДИРОВАНИЕ

1.1 Задание

Реализовать знаковое кодирование и декодирование текстового сообщения. Алфавит сообщения состоит из 64 символов: латинские буквы (52 символа), цифры (0...9), пробел и точка. Длина сообщения от 30 до 100 символов. Запрещено использовать встроенные функции Matlab/Octave для преобразования формата данных.

1.2 Теоретическое описание

Знаковое кодирование — это процесс преобразования символьного сообщения в двоичную последовательность. Каждому символу ставится в соответствие уникальное кодовое слово фиксированной длины. Для алфавита из 64 символов достаточно 6 бит ($2^6 = 64$), однако в данной работе используется 8-битное кодирование для удобства.

1.3 Реализация

Кодирование выполняется по следующему правилу:

- A-Z: коды 0-25
- a-z: коды 26-51
- 0-9: коды 52-61
- пробел: код 62
- точка: код 63

Декодер выполняет обратное преобразование: группирует биты по 8, преобразует в число и по таблице символов восстанавливает текст.

```
1 class SignCoder:
2     BITS = 8
3
4     def sign_encoder(text):
5         if not text or len(text) < 30 or len(text) > 100:
6             return None
```

```

7     bits = []
8     for c in text:
9         if 'A' <= c <= 'Z':
10            code = ord(c) - ord('A')
11        elif 'a' <= c <= 'z':
12            code = ord(c) - ord('a') + 26
13        elif '0' <= c <= '9':
14            code = ord(c) - ord('0') + 52
15        elif c == ' ':
16            code = 62
17        elif c == '.':
18            code = 63
19        else:
20            return None
21        bits.append(format(code, '08b'))
22    return ''.join(bits)
23
24    def sign_decoder(bits):
25        if not bits or len(bits) % 8 != 0:
26            return None
27        text = []
28        chars =
29            'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789
30            .'
31        for i in range(0, len(bits), 8):
32            code = int(bits[i:i + 8], 2)
33            if code < 64:
34                text.append(chars[code])
35            else:
36                return None
37        return ''.join(text)

```

Листинг 1.1 — Знаковый кодер и декодер

1.4 Результат

Исходное сообщение *"Hello World. This is test message and no more..."* длиной 51 символ было преобразовано в 408 бит. При обратном декодировании сообщение восстановлено полностью без ошибок.

Исходное сообщение: Hello World. This is test message and no more...
 Длина сообщения: 51 символов

Символьное кодирование: 408 бит

Полученное сообщение: Hello World. This is test message and

1.5 Вывод

Знаковый кодер и декодер успешно реализованы. Преобразование текста в битовую последовательность и обратно выполняется корректно.

ПРАКТИКА 2. ПОМЕХОУСТОЙЧИВОЕ КОДИРОВАНИЕ

2.1 Задание

Реализовать помехоустойчивое кодирование с использованием кода Хэмминга.

2.2 Теоретическое описание

Код Хэмминга — это линейный блочный код, позволяющий обнаруживать и исправлять одиночные ошибки. Для кода Хэмминга с k информационными битами количество контрольных битов m определяется из условия:

$$2^m \geq k + m + 1$$

Общая длина кодового слова: $n = k + m$. Контрольные биты располагаются на позициях, являющихся степенями двойки (1, 2, 4, 8, ...). Каждый контрольный бит вычисляется как XOR (сложение по модулю 2) информационных битов, номера позиций которых имеют единицу в соответствующем разряде двоичного представления.

2.3 Реализация

```
1 class HammingCoder:
2     def __init__(self, k_bits):
3         self.K = k_bits
4         self.m = 0
5         while (2 ** self.m) < (self.K + self.m + 1):
6             self.m += 1
7         self.N = self.K + self.m
8
9     def encode(self, bits):
10        if not bits:
11            return bits
12        remainder = len(bits) % self.K
13        if remainder:
14            bits = bits + '0' * (self.K - remainder)
15        encoded = []
```

```

16     for i in range(0, len(bits), self.K):
17         data = [int(b) for b in bits[i:i + self.K]]
18         codeword = [0] * self.N
19         data_idx = 0
20         for pos in range(1, self.N + 1):
21             if not (pos & (pos - 1) == 0):
22                 codeword[pos - 1] = data[data_idx]
23                 data_idx += 1
24         for j in range(self.m):
25             p_pos = 2 ** j
26             parity = 0
27             for pos in range(1, self.N + 1):
28                 if pos & p_pos:
29                     parity ^= codeword[pos - 1]
30             codeword[p_pos - 1] = parity
31         encoded.extend([str(b) for b in codeword])
32     return ''.join(encoded)
33
34 def decode(self, bits):
35     if not bits or len(bits) % self.N != 0:
36         return bits if not bits else None
37     decoded = []
38     for i in range(0, len(bits), self.N):
39         r = [int(b) for b in bits[i:i + self.N]]
40         syndrome = 0
41         for j in range(self.m):
42             p_pos = 2 ** j
43             parity = 0
44             for pos in range(1, self.N + 1):
45                 if pos & p_pos:
46                     parity ^= r[pos - 1]
47             if parity:
48                 syndrome += p_pos
49         if 0 < syndrome <= self.N:
50             r[syndrome - 1] ^= 1
51         for pos in range(1, self.N + 1):
52             if not (pos & (pos - 1) == 0):
53                 decoded.append(str(r[pos - 1]))
54     return ''.join(decoded)

```

Листинг 2.1 — Кодек Хэмминга

2.4 Результат

При использовании кода Хэмминга с 11 информационными битами:

- Количество контрольных битов: $m = 4$
- Длина кодового слова: $n = 15$
- Скорость кодирования: $R = 11/15 \approx 0.733$

Исходное сообщение из 408 бит после кодирования занимает 570 бит.

2.5 Вывод

Код Хэмминга успешно реализован. Он позволяет исправлять одиночные ошибки, что повышает помехоустойчивость передачи данных.

ПРАКТИКА 3. ПЕРЕМЕЖЕНИЕ

3.1 Задание

Реализовать операцию прямого и обратного перемежения закодированного битового сообщения.

3.2 Теоретическое описание

Перемежение (interleaving) — это процесс перестановки битов в кодовом слове по определенному закону. Основная цель перемежения — борьба с пакетными ошибками. При возникновении пакетной ошибки в канале после обратного перемежения ошибки распределяются по всему кодовому слову, что позволяет эффективно исправлять их с помощью помехоустойчивого кода.

3.3 Реализация

Перемежитель использует псевдослучайную перестановку бит на основе заданного seed. Для обратного перемежения используется сохраненный закон перестановки.

```
1 class Interleaver:
2     def __init__(self, seed=42):
3         self.seed = seed
4         self.permutation = None
5
6     def interleave(self, bits):
7         if not bits:
8             return bits
9         np.random.seed(self.seed)
10        n = len(bits)
11        self.permutation = np.random.permutation(n)
12        interleaved = ['0'] * n
13        for i, pos in enumerate(self.permutation):
14            interleaved[pos] = bits[i]
15        return ''.join(interleaved)
16
17 class Deinterleaver:
```

```

18     def __init__(self, interleaver):
19         self.permutation = interleaver.permutation
20
21     def deinterleave(self, bits):
22         if not bits or self.permutation is None:
23             return bits
24         n = len(bits)
25         deinterleaved = ['0'] * n
26         for i, pos in enumerate(self.permutation):
27             deinterleaved[i] = bits[pos]
28         return ''.join(deinterleaved)

```

Листинг 3.1 — Перемежитель и деперемежитель

3.4 Результат

Перемежение:

После перемежения: 570 бит

Первые 10 бит до перемежения: 1011000101

Первые 10 бит после перемежения: 0010111010

При обратном перемежении битовая последовательность полностью восстанавливается.

3.5 Вывод

Перемежитель и деперемежитель успешно реализованы. Псевдослучайная перестановка бит позволяет бороться с пакетными ошибками.

ПРАКТИКА 4. QPSK МОДУЛЯЦИЯ

4.1 Задание

Реализовать операцию QPSK модуляции битового сообщения и QPSK демодуляции символов.

4.2 Теоретическое описание

QPSK (Quadrature Phase Shift Keying) — метод фазовой модуляции, при котором фаза несущего сигнала принимает одно из четырех возможных значений. Каждому значению фазы соответствует пара бит (дибит). Скорость передачи символов в два раза ниже скорости передачи бит.

Таблица QPSK модуляции:

Дибит	I	Q	Символ
00	+0.707	+0.707	$0.707 + 0.707j$
01	+0.707	-0.707	$0.707 - 0.707j$
10	-0.707	+0.707	$-0.707 + 0.707j$
11	-0.707	-0.707	$-0.707 - 0.707j$

4.3 Реализация

```
1 class Modulator:
2     def modulate(bits):
3         if not bits:
4             return []
5         if len(bits) % 2 != 0:
6             bits = bits + '0'
7         symbols = []
8         for i in range(0, len(bits), 2):
9             bit_pair = bits[i:i + 2]
10            if bit_pair == '00':
11                symbols.append(complex(0.707, 0.707))
12            elif bit_pair == '01':
13                symbols.append(complex(0.707, -0.707))
14            elif bit_pair == '10':
15                symbols.append(complex(-0.707, 0.707))
```

```

16         else:
17             symbols.append(complex(-0.707, -0.707))
18     return symbols
19
20 class Demodulator:
21     def demodulate(symbols):
22         if not symbols:
23             return ""
24         bits = []
25         for symbol in symbols:
26             if symbol.real > 0 and symbol.imag > 0:
27                 bits.append('00')
28             elif symbol.real > 0 and symbol.imag < 0:
29                 bits.append('01')
30             elif symbol.real < 0 and symbol.imag > 0:
31                 bits.append('10')
32             else:
33                 bits.append('11')
34     return ''.join(bits)

```

Листинг 4.1 — QPSK модулятор и демодулятор

4.4 Результат

QPSK модуляция:

До модуляции: 570 бит

После модуляции: 285 символов

Демодуляция выполняется путем определения области комплексной плоскости, в которую попал принятый символ.

4.5 Вывод

QPSK модулятор и демодулятор успешно реализованы. При отсутствии шума демодуляция выполняется без ошибок.

ПРАКТИКА 5. OFDM МОДУЛЯЦИЯ

5.1 Задание

Реализовать формирование OFDM символа, включающего QPSK символы, опорные сигналы, нулевые защитные интервалы и циклический префикс.

5.2 Теоретическое описание

OFDM (Orthogonal Frequency Division Multiplexing) — метод мультиплексирования, при котором данные передаются на множестве ортогональных поднесущих. Основные элементы OFDM символа:

1. **Информационные поднесущие** — передают QPSK символы
2. **Опорные сигналы (пилоты)** — известные символы для оценки канала
3. **Нулевые поднесущие** — защитные интервалы для борьбы с внеполосным излучением
4. **Циклический префикс (CP)** — копия конца символа, добавляемая в начало для борьбы с межсимвольной интерференцией

Количество опорных поднесущих:

$$N_{RS} = \left\lfloor \frac{N_{QPSK}}{\Delta RS} \right\rfloor$$

Количество нулевых поднесущих:

$$N_2 = C \cdot (N_{RS} + N_{QPSK})$$

5.3 Реализация

```
1 class OfdmModulator:
2     def __init__(self, delta_rs=6, cp_len=16, c_param=0.25,
3         pilot_value=complex(0.707, 0.707)):
4         self.delta_rs = delta_rs
```

```

4     self.cp_len = cp_len
5     self.c_param = c_param
6     self.pilot_value = pilot_value
7
8     def modulate(self, symbols):
9         self.n_qpsk = len(symbols)
10        self.n_pilots = int(np.floor(self.n_qpsk / self.delta_rs))
11        if self.n_pilots == 0:
12            self.n_pilots = 1
13        self.n_zeros = int(self.c_param * (self.n_pilots +
14            self.n_qpsk))
15        self.total_carriers = self.n_pilots + self.n_qpsk + 2 *
16            self.n_zeros
17
18        spectrum = np.zeros(self.total_carriers, dtype=complex)
19
20        self.pilot_indices = []
21        pilot_pos = self.n_zeros
22        for i in range(self.n_pilots):
23            idx = pilot_pos + i * self.delta_rs
24            if idx < self.total_carriers - self.n_zeros:
25                self.pilot_indices.append(int(idx))
26                spectrum[idx] = self.pilot_value
27
28        self.data_indices = []
29        data_idx = 0
30        for i in range(self.total_carriers):
31            if i >= self.n_zeros and i < self.total_carriers -
32                self.n_zeros:
33                if i not in self.pilot_indices:
34                    if data_idx < len(symbols):
35                        spectrum[i] = symbols[data_idx]
36                        self.data_indices.append(i)
37                        data_idx += 1
38
39        time_signal = np.fft.ifft(spectrum)
40        cp = time_signal[-self.cp_len:]
41        ofdm_symbol = np.concatenate([cp, time_signal])
42        return ofdm_symbol

```

Листинг 5.1 — OFDM модулятор

5.4 Результат

При параметрах: $\Delta RS = 6$, $T_{CP} = 16$, $C = 0.25$, $N_{QPSK} = 285$:

- $N_{RS} = 47$ опорных поднесущих
- $N_2 = 83$ нулевых поднесущих
- Общее количество поднесущих: 498
- Длина OFDM символа: $498 + 16 = 514$ отсчетов

5.5 Вывод

OFDM модулятор успешно реализован. Он формирует OFDM символ с учетом всех необходимых компонентов: опорные сигналы, информационные данные, нулевые поднесущие и циклический префикс.

ПРАКТИКА 6. МОДЕЛЬ КАНАЛА ПЕРЕДАЧИ

6.1 Задание

Реализовать модель многолучевого канала с аддитивным белым гауссовым шумом.

6.2 Теоретическое описание

Модель многолучевого канала учитывает распространение сигнала по нескольким путям (лучам). Каждый луч характеризуется:

- Задержкой τ_i относительно прямого луча
- Ослаблением G_i , зависящим от расстояния

Задержка рассчитывается по формуле:

$$\tau_i = \frac{D_i - D_1}{c \cdot T_s}, \quad i = 1, 2, \dots, N_B$$

Ослабление сигнала в вакууме:

$$G_i = \frac{c}{4\pi D_i f_0}$$

После суммирования всех лучей добавляется аддитивный белый гауссов шум (АБГШ) с заданной спектральной плотностью мощности N_0 .

6.3 Реализация

```
1 class MultipathChannel:
2     def __init__(self, fc=2.4e9, bandwidth=9e6, num_paths=3,
3         n0_db=-100):
4         self.fc = fc
5         self.bandwidth = bandwidth
6         self.num_paths = num_paths
7         self.n0_db = n0_db
8         self.c = 3e8
9         self.Ts = 1.0 / bandwidth
10    def propagate(self, tx_signal):
```

```

11     distances = np.random.uniform(10, 500, self.num_paths)
12     min_dist = np.min(distances)
13     delays = []
14     for d in distances:
15         tau = (d - min_dist) / (self.c * self.Ts)
16         delays.append(int(round(tau)))
17     gains = []
18     for d in distances:
19         gain = self.c / (4 * np.pi * d * self.fc)
20         gains.append(gain)
21     L = len(tx_signal)
22     rx_sum = np.zeros(L + max(delays), dtype=complex)
23     for i in range(self.num_paths):
24         shift = delays[i]
25         gain = gains[i]
26         shifted = np.zeros(L + shift, dtype=complex)
27         shifted[shift:] = tx_signal * gain
28         rx_sum[:len(shifted)] += shifted
29     rx_signal = rx_sum[:L]
30     n0_linear = 10 ** (self.n0_db / 10.0)
31     noise_power = n0_linear * self.bandwidth
32     noise_std = np.sqrt(noise_power / 2.0)
33     noise = noise_std * (np.random.randn(L) + 1j *
34                          np.random.randn(L))
35     rx_signal += noise
36     return rx_signal

```

Листинг 6.1 — Модель многолучевого канала

6.4 Результат

При количестве лучей $N_B = 3$ и мощности шума $N_0 = -100$ дБ сигнал на выходе канала имеет ту же длину, что и на входе, но содержит искажения от многолучевости и шума.

6.5 Вывод

Модель многолучевого канала с АБГШ успешно реализована. Она учитывает задержки и ослабления для каждого луча, а также аддитивный шум.

ПРАКТИКА 7. ЭКВАЛАЙЗИРОВАНИЕ И OFDM ДЕМОДУЛЯЦИЯ

7.1 Задание

Реализовать OFDM демодуляцию, оценку АЧХ канала и эквалайзирование.

7.2 Теоретическое описание

Эквалайзер предназначен для компенсации искажений, внесенных каналом передачи. Основные этапы:

1. **Удаление циклического префикса** — отбрасывание первых T_{CP} отсчетов
2. **Прямое преобразование Фурье (FFT)** — переход в частотную область
3. **Извлечение опорных сигналов** — выделение пилот-символов
4. **Оценка АЧХ канала** — $H = R_{rx}/R_{tx}$
5. **Интерполяция** — линейная интерполяция H на все поднесущие
6. **Эквалайзирование** — $C_{EQ} = C/H_{EQ}$

7.3 Реализация

```
1 class OfdmDemodulator:
2     def __init__(self, modulator):
3         self.delta_rs = modulator.delta_rs
4         self.cp_len = modulator.cp_len
5         self.pilot_value = modulator.pilot_value
6         self.pilot_indices = modulator.pilot_indices
7         self.data_indices = modulator.data_indices
8         self.n_qpsk = modulator.n_qpsk
9         self.n_zeros = modulator.n_zeros
10        self.total_carriers = modulator.total_carriers
11
12    def demodulate(self, ofdm_signal):
13        time_signal = ofdm_signal[self.cp_len:self.cp_len +
14                                   self.total_carriers]
15        spectrum = np.fft.fft(time_signal)
```

```

15
16     rx_pilots = np.array([spectrum[idx] for idx in
17                           self.pilot_indices])
18     tx_pilots = np.array([self.pilot_value] *
19                           len(self.pilot_indices))
20     h_est = rx_pilots / tx_pilots
21
22     all_data_indices = list(range(self.n_zeros,
23                                   self.total_carriers - self.n_zeros))
24     h_full = np.interp(all_data_indices, self.pilot_indices, h_est)
25     heq = 1.0 / h_full
26
27     recovered_symbols = []
28     for i, idx in enumerate(self.data_indices):
29         if idx < len(spectrum):
30             eq_idx = all_data_indices.index(idx) if idx in
31                     all_data_indices else i
32             if eq_idx < len(heq):
33                 recovered_symbols.append(spectrum[idx] *
34                                         heq[eq_idx])
35     return np.array(recovered_symbols[:self.n_qpsk])

```

Листинг 7.1 — OFDM демодулятор с эквалайзером

7.4 Результат

После эквалайзирования искажения, внесенные многолучевым каналом, компенсируются. Восстановленные QPSK символы близки к переданным.

7.5 Вывод

OFDM демодулятор с эквалайзером успешно реализован. Оценка АЧХ канала по пилот-сигналам и последующая коррекция позволяют компенсировать искажения.

ПРАКТИКА 8. РАСЧЕТ BER И ПОСТРОЕНИЕ ГРАФИКОВ

8.1 Задание

Реализовать расчет вероятности битовых ошибок (BER) и построение графиков спектров и сигнальных созвездий.

8.2 Теоретическое описание

Вероятность битовой ошибки (Bit Error Rate, BER) вычисляется по формуле:

$$BER = \frac{\text{количество ошибочных бит}}{\text{общее количество бит}}$$

Сравнение производится между битовым сообщением после знакового кодера (на передатчике) и перед знаковым декодером (в приемнике).

8.3 Реализация

8.3.1 Расчет BER

```
1 def calculate_ber(tx_bits, rx_bits):
2     if len(tx_bits) != len(rx_bits):
3         min_len = min(len(tx_bits), len(rx_bits))
4         tx_bits = tx_bits[:min_len]
5         rx_bits = rx_bits[:min_len]
6     errors = sum(1 for i in range(len(tx_bits)) if tx_bits[i] !=
7                 rx_bits[i])
8     ber = errors / len(tx_bits) if len(tx_bits) > 0 else 0
9     return ber, errors
```

Листинг 8.1 — Расчет BER

8.3.2 Построение графиков

```
1 def plot_spectrums(tx_spectrum, rx_spectrum_before, rx_spectrum_after):
2     plt.figure(figsize=(12, 8))
3
```

```

4 plt.subplot(3, 1, 1)
5 plt.plot(np.abs(tx_spectrum))
6 plt.title('Спектр переданного OFDM символа')
7 plt.xlabel('Индекс поднесущей')
8 plt.ylabel('Амплитуда')
9 plt.grid(True)
10
11 plt.subplot(3, 1, 2)
12 plt.plot(np.abs(rx_spectrum_before))
13 plt.title('Спектр принятого OFDM символа до эквалайзирования ')
14 plt.xlabel('Индекс поднесущей')
15 plt.ylabel('Амплитуда')
16 plt.grid(True)
17
18 plt.subplot(3, 1, 3)
19 plt.plot(np.abs(rx_spectrum_after))
20 plt.title('Спектр принятого OFDM символа после эквалайзирования ')
21 plt.xlabel('Индекс поднесущей')
22 plt.ylabel('Амплитуда')
23 plt.grid(True)
24
25 plt.tight_layout()
26 plt.show()

```

Листинг 8.2 — Построение графиков спектров

```

1 def plot_constellations(tx_symbols, rx_symbols):
2     plt.figure(figsize=(12, 5))
3
4     plt.subplot(1, 2, 1)
5     plt.scatter([s.real for s in tx_symbols], [s.imag for s in
6         tx_symbols])
7     plt.title('Сигнальное созвездие QPSK в передатчике ')
8     plt.xlabel('I')
9     plt.ylabel('Q')
10    plt.grid(True)
11    plt.axis('equal')
12
13    plt.subplot(1, 2, 2)
14    plt.scatter([s.real for s in rx_symbols], [s.imag for s in
15        rx_symbols])
16    plt.title('Сигнальное созвездие QPSK в приемнике ')
17    plt.xlabel('I')
18    plt.ylabel('Q')
19    plt.grid(True)

```

```
18 plt.axis('equal')
19
20 plt.tight_layout()
21 plt.show()
```

Листинг 8.3 — Построение сигнальных созвездий

8.4 Результат

BER = 0.000000

Ошибочных бит: 0 из 408

На графиках отображаются:

- Спектр переданного OFDM символа
- Спектр принятого сигнала до эквалайзирования (с искажениями)
- Спектр принятого сигнала после эквалайзирования (восстановленный)
- Сигнальное созвездие QPSK в передатчике (4 точки)
- Сигнальное созвездие QPSK в приемнике (облака точек из-за шума)

8.5 Вывод

Расчет BER и построение графиков успешно реализованы. При заданных параметрах канала ($N_B = 3$, $N_0 = -100$ дБ) ошибок при передаче не обнаружено. Графики наглядно демонстрируют эффект эквалайзирования.